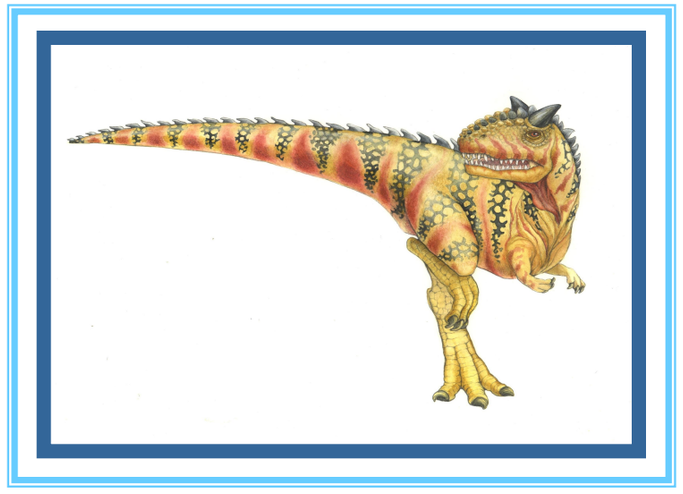
Chapter 4: Threads

Shatabdi Roy Moon
Lecturer, dept. of CSE



What is Thread in OS?

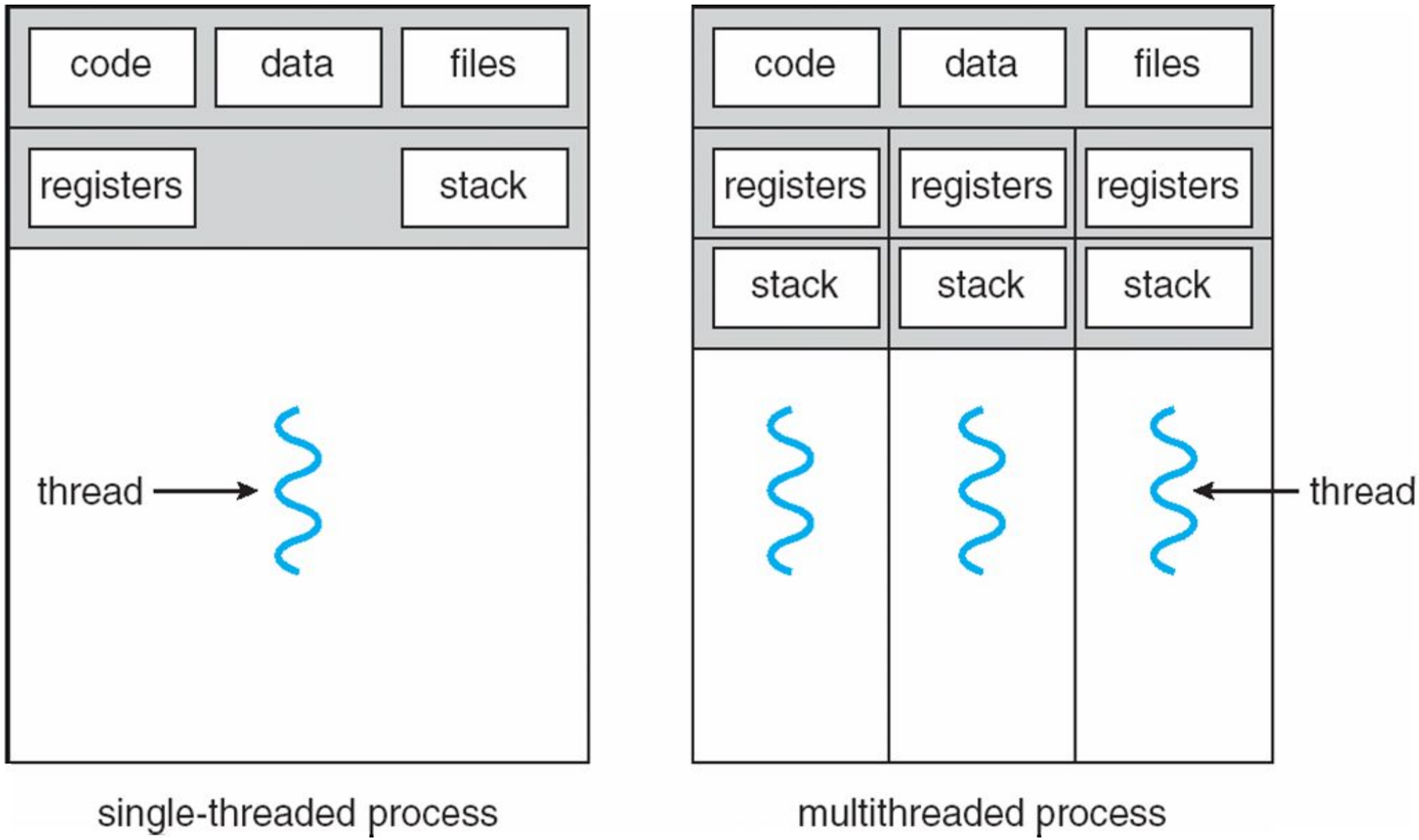
- **Thread is a sequential flow of tasks within a process.** Threads in OS can be of the same or different types. Threads are used to increase the performance of the applications.
- Each thread has its own program counter, stack, and set of registers. But the threads of a single process might share the same code and data/file. **Threads are also termed as lightweight processes as they share common resources**.
- **Example:** While playing a movie on a device the audio and video are controlled by different threads in the background.

Why do we need Threads?

Threads in the operating system provide multiple benefits and improve the overall performance of the system. Some of the reasons threads are needed in the operating system are:

- Since threads use the same data and code, the operational cost between threads is low.
- Creating and terminating a thread is faster compared to creating or terminating a process.
- Context switching is faster in threads compared to processes.

Single and Multithreaded Processes



single-threaded process

multithreaded process

Why Multithreading?

In Multithreading, the idea is to divide a single process into multiple threads instead of creating a whole new process. Multithreading is done to achieve parallelism and to improve the performance of the applications as it is faster in many ways which were discussed above. The other advantages of multithreading are mentioned below.

- **Resource Sharing:** Threads of a single process share the same resources such as code, data/file.
- **Responsiveness:** Program responsiveness enables a program to run even if part of the program is blocked or executing a lengthy operation. Thus, increasing the responsiveness to the user.
- **Economy:** It is more economical to use threads as they share the resources of a single process. On the other hand, creating processes is expensive.
- **Utilization of multiprocessor architectures / Scalability:** A single threaded process can only run on one CPU, no matter how many may be available, whereas the execution of a multithreaded application may be split amongst available processors.

Programming Challenges

- For application programmers, there are five areas where multicore chips present new challenges:
  1. **Identifying tasks -** Examining applications to find activities that can be performed concurrently.
  2. **Balance Finding** - Finding tasks to run concurrently that provide equal value.
  3. **Data splitting –** To prevent the threads from interfering with one another.
  4. **Data dependency** – If one task is dependent upon the results of another, then the tasks need to be synchronized to assure access in the proper order.
  5. **Testing and debugging** – Inherently more difficult in parallel processing situations, as the race conditions become much more complex and difficult to identify.

Process vs Thread

- **Process simply means any program in execution while the thread is a segment of a process**. The main differences between process and thread are mentioned below:

Process	Thread
Processes use more resources and hence they are termed as heavyweight processes.	Threads share resources and hence they are termed as lightweight processes.
Creation and termination times of processes are slower.	Creation and termination times of threads are faster compared to processes.
Processes have their own code and data/file.	Threads share code and data/file within a process.
Communication between processes is slower.	Communication between threads is faster.
Context Switching in processes is slower.	Context switching in threads is faster.
Processes are independent of each other.	Threads, on the other hand, are interdependent. (i.e they can read, write or change another thread's data)
Eg: Opening two different browsers.	Eg: Opening two tabs in the same browser.

Multicore Programming

- A recent trend in computer architecture is to produce chips with multiple cores, or CPUs on a single chip.
- A multithreaded application running on a traditional single core chip would have to interleave the threads. On a multicore chip, however, the threads could be spread across the available cores, allowing true parallel processing.

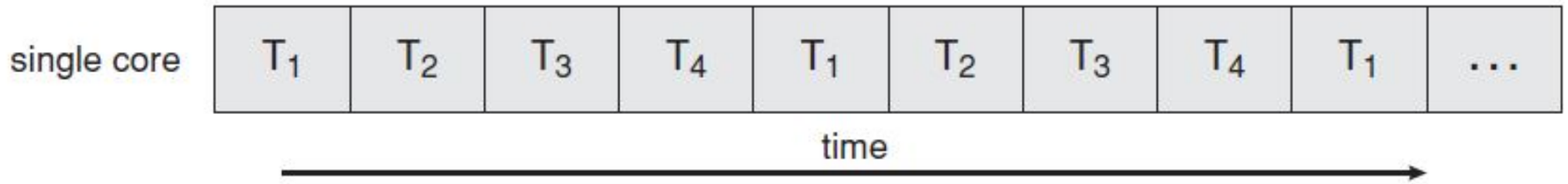


Figure 4.3 Concurrent execution on a single-core system.

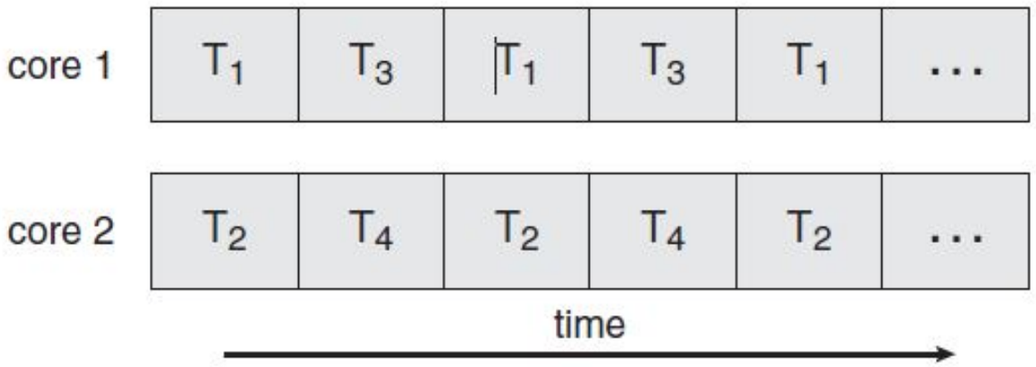


Figure 4.4 Parallel execution on a multicore system.

Multithreading Models

- There are **two** types of threads to be managed in a modern system: **User threads** and **kernel threads**.
- **User threads** are supported above the kernel, without kernel support. These are the threads that application programmers would put into their programs.
- **Kernel threads** are supported within the kernel of the OS itself. All modern OSes support kernel level threads, allowing the kernel to perform multiple simultaneous tasks and/or to service multiple kernel system calls simultaneously.
- In a specific implementation, the user threads must be mapped to kernel threads, using one of the following strategies.

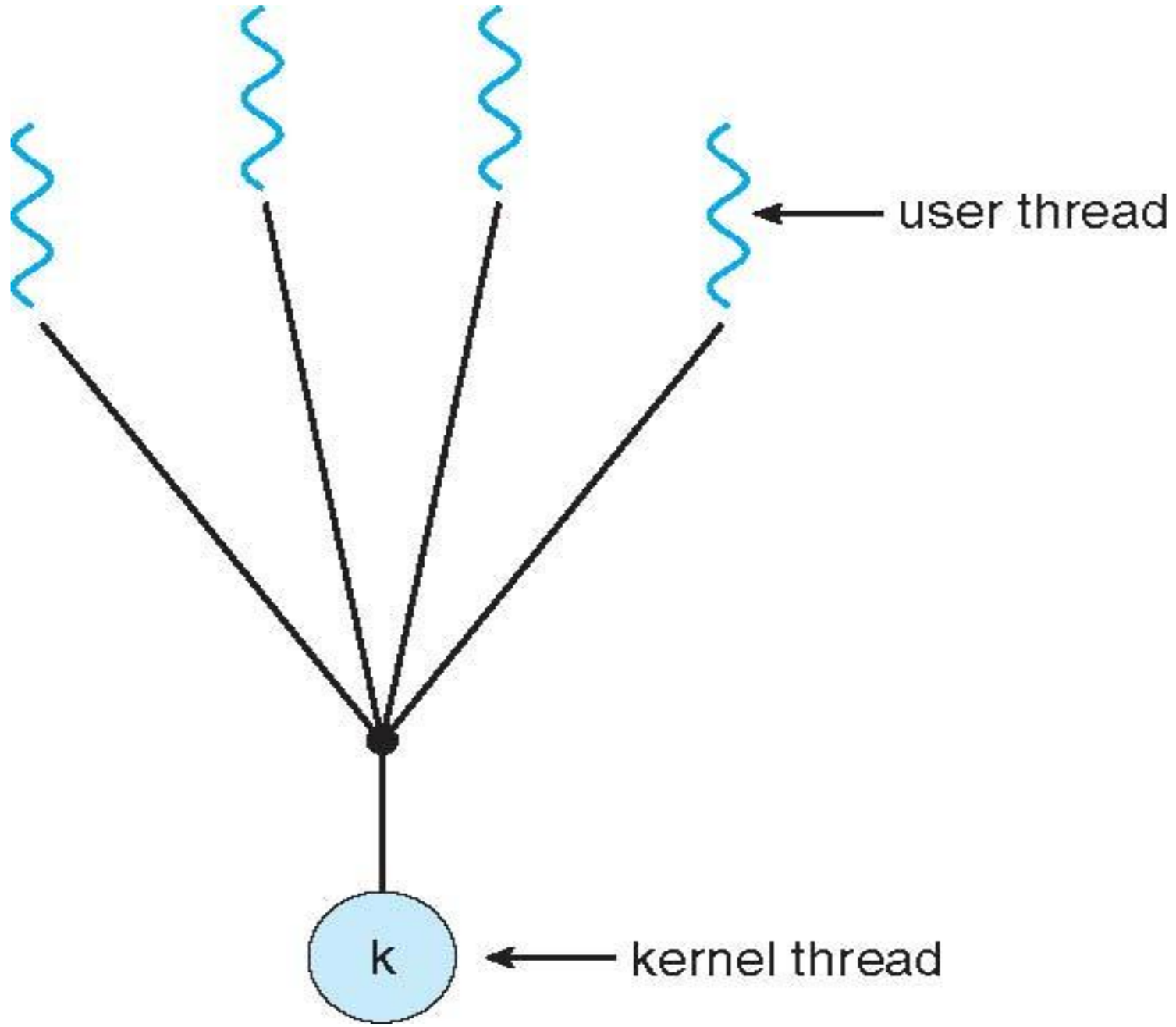
Multithreading Models

- **Many-to-One**
- **One-to-One**
- **Many-to-Many**

Many-to-One Model

- In the many to one model, many user level threads are all mapped onto a single kernel thread.
- Thread management is handled by the thread library in user space, which is very efficient.
- However, if a blocking system call is made, then the entire process blocks, even if the other user threads would otherwise be able to continue.
- Because a single kernel thread can operate only on a single CPU, the many to one model does not allow individual processes to be split across multiple CPUs.
- Green threads for Solaris and GNU Portable Threads implement the many to one model in the past, but few systems continue to do so today.

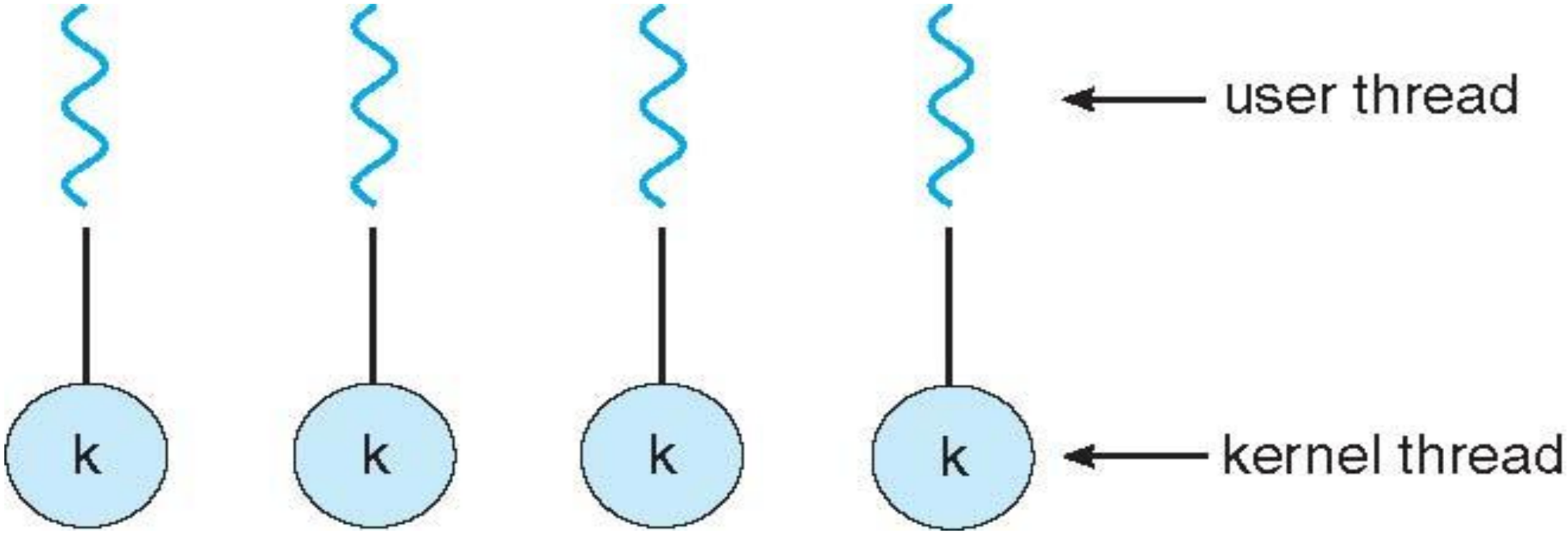
Many-to-One Model



One-to-One Model

- The one to one model creates a separate kernel thread to handle each user thread.
- One to one model overcomes the problems listed above involving blocking system calls and the splitting of processes across multiple CPUs.
- However the overhead of managing the one to one model is more significant, involving more overhead and slowing down the system.
- Most implementations of this model place a limit on how many threads can be created.
- Linux and Windows from 95 to XP implement the one to one model for threads.

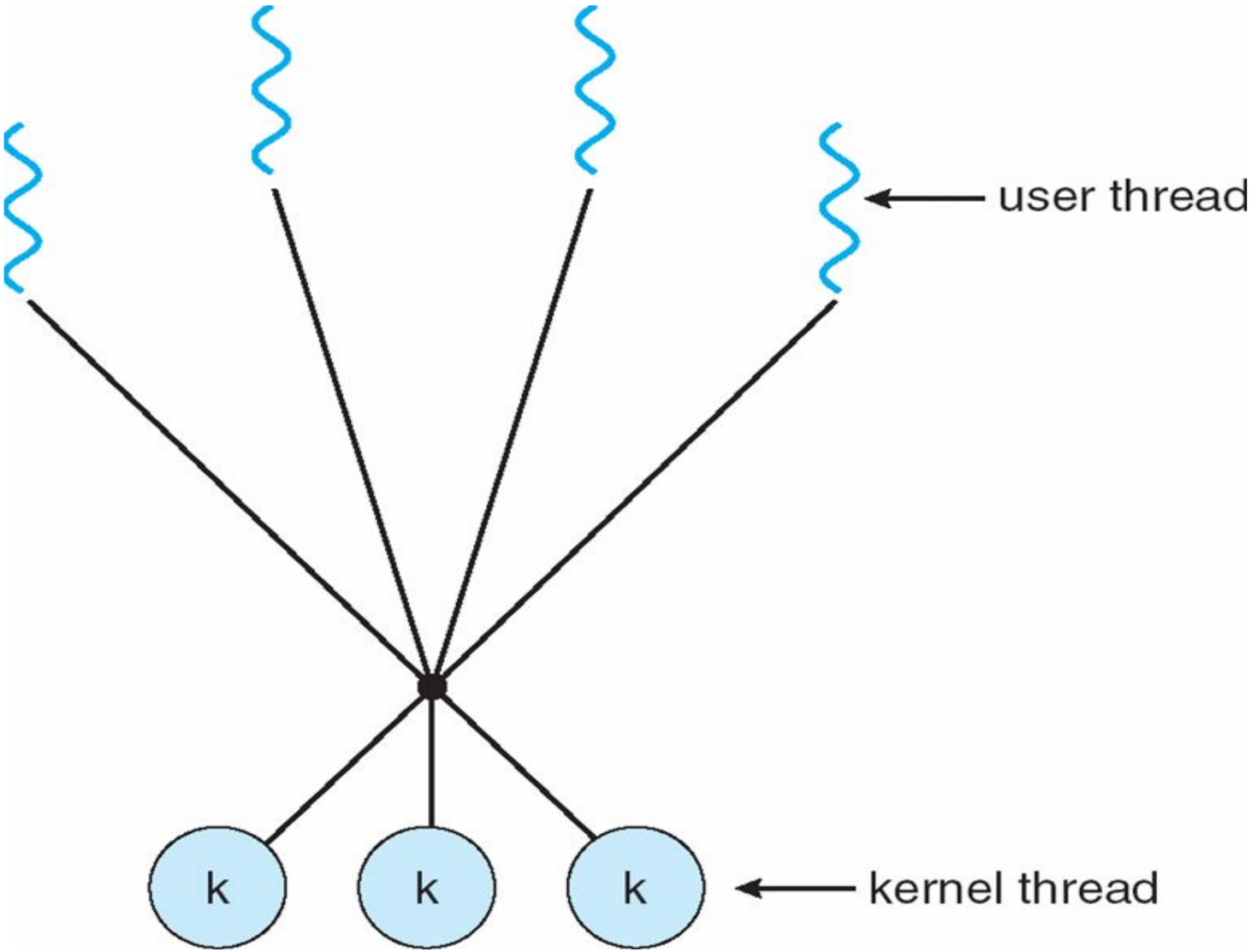
One-to-one Model



Many-to-Many Model

- The many to many model multiplexes any number of user threads onto an equal or smaller number of kernel threads, combining the best features of the one to one and many to one models.
- Users have no restrictions on the number of threads created.
- Blocking kernel system calls do not block the entire process.
- Processes can be split across multiple processors.
- Individual processes may be allocated variable numbers of kernel threads, depending on the number of CPUs present and other factors.

Many-to-Many Model



Thread Libraries

- Provides the programmer and API for creating and managing threads.
- Three primary thread libraries:
  - POSIX Pthreads
  - Win32 threads
  - Java threads

Threading Issues

There are a number of issues that arise with threading. Some of them are mentioned below:

- **The semantics of fork() and exec() system calls:**

  The fork() call is used to create a duplicate child process. During a fork() call the issue that arises is whether the whole process should be duplicated or just the thread which made the fork() call should be duplicated. **The exec() call replaces the whole process that called it including all the threads in the process with a new program**.

- **Thread cancellation:**

  The termination of a thread before its completion is called thread cancellation and the terminated thread is termed as target thread. Thread cancellation is of two types:

  1. **Asynchronous Cancellation:** In asynchronous cancellation, one thread immediately terminates the target thread.
  2. **Deferred Cancellation:** In deferred cancellation, the target thread periodically checks if it should be terminated.

Threading Issues

- **Signal handling:**

  In UNIX systems, a signal is used to notify a process that a particular event has happened. Based on the source of the signal, signal handling can be categorized as:

  1. **Asynchronous Signal:** The signal which is generated outside the process which receives it.
  2. **Synchronous Signal:** The signal which is generated and delivered in the same process.

End of Chapter 4